



温州肯恩大学
WENZHOU-KEAN UNIVERSITY

MGS 3001 WHS01
Python Programming for Business

Functions in Python

Dawei Chen

Week 3-2, 12 March 2026

Recap

- Python Data Structures
 - **List** → ordered and mutable collection
 - **Tuple** → ordered but immutable record
 - **Set** → unordered collection of unique elements
 - **Dictionary** → key-value mapping for structured data
- Control Flow in Python
 - **Decision making** → if, elif, else
 - **Iteration** → for, while
 - **Loop control** → break, continue, pass



Learning Objectives

By the end of today's session, you should be able to:

- Explain what a **function** is and why it is useful
- Define a function using `def`
- Pass `arguments` using positional, keyword, and default parameters
- Understand flexible arguments using `*args` and `**kwargs`
- Interpret `return values` and use them in further computations
- Recognize and apply `lambda functions` for simple operations

Motivation

Scenario: Suppose a company sells products on an e-commerce platform. For each order, we need to compute several metrics:

- Revenue
- Discount amount
- Final price paid by the customer

A student might initially write code like this:

Potential issues

- Duplicated code
- Harder to modify
- Higher chance of mistakes
- Reduced readability and maintainability

```
price = 120
quantity = 3
discount_rate = 0.1

revenue = price * quantity
discount = revenue * discount_rate
final_payment = revenue - discount

print(revenue, discount, final_payment)

price = 80
quantity = 5
discount_rate = 0.1

revenue = price * quantity
discount = revenue * discount_rate
final_payment = revenue - discount

print(revenue, discount, final_payment)

price = 200
quantity = 2
discount_rate = 0.1

revenue = price * quantity
discount = revenue * discount_rate
final_payment = revenue - discount

print(revenue, discount, final_payment)
```

What do you think about this program?

What if we have 10,000 orders?

Better Solution: Use a Function

Now the program becomes **cleaner**

```
def order_summary(price, quantity, discount_rate):  
    revenue = price * quantity  
    discount = revenue * discount_rate  
    final_payment = revenue - discount  
    return revenue, discount, final_payment  
  
print(order_summary(120, 3, 0.1))  
print(order_summary(80, 5, 0.1))  
print(order_summary(200, 2, 0.1))
```

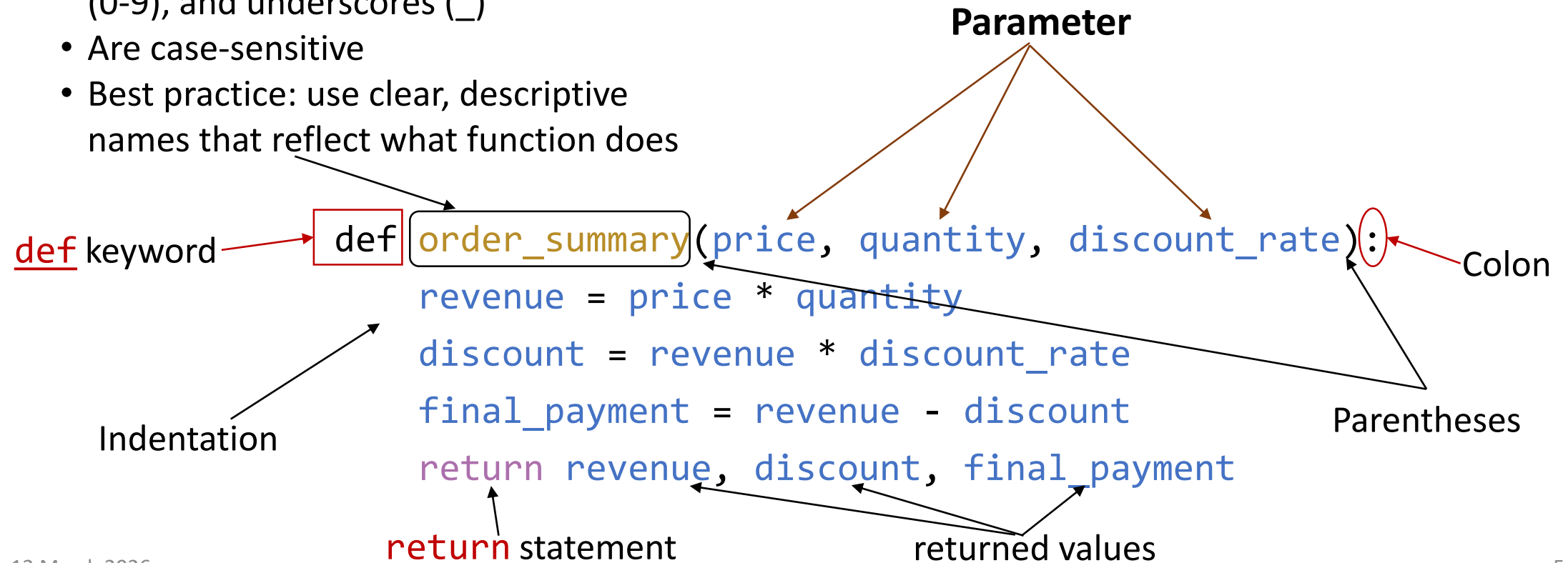
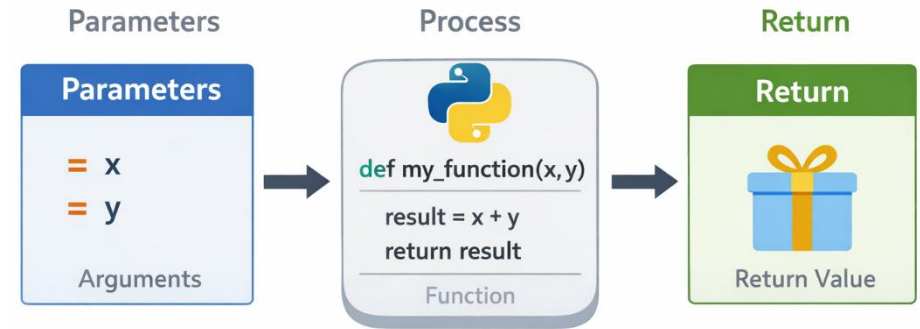
What if we have 10,000 orders?

- Functions allow us to
 - Encapsulate reusable logic
 - Avoid repeating code
 - Improve readability and maintainability
- Programming principle
 - Break **complex problems** into **small, reusable functions**

Basic Function Syntax

Function Name

- Must start with a letter or underscore
- Can contain letters (A-Z, a-z), numbers (0-9), and underscores (_)
- Are case-sensitive
- Best practice: use clear, descriptive names that reflect what function does



Arguments

- Information passed into a function
 - **Parameters:** **Variable** listed in the parentheses of a function **definition**
 - **Arguments:** **Actual values** passed to the function when it is **called**
- What happens if we call the `order_summary` function in the following ways? Try each case and summarize your findings.

```
def order_summary(price, quantity, discount_rate):  
    revenue = price * quantity  
    discount = revenue * discount_rate  
    final_payment = revenue - discount  
    return revenue, discount, final_payment  
  
print(order_summary(120, 3, 0.1))  
print(order_summary(80, 5, 0.1))  
print(order_summary(200, 2, 0.1))
```

```
order_summary(120, 3, 0.1)  
  
order_summary(3, 0.1, 120)  
  
order_summary(120, 3)  
  
order_summary("120", 3, 0.1)  
  
order_summary(price=120, quantity=3, discount_rate=0.1)  
order_summary(discount_rate=0.1, price=120, quantity=3)  
  
order_summary(120, quantity=3, discount_rate=0.1)  
  
order_summary(quantity=3, 120, discount_rate=0.1)
```

Arguments

- The **order** of arguments matter (**positional arguments**)

```
order_summary(3, 0.1, 120)
```

- The **number** of arguments **must match** the parameters

```
order_summary(120, 3)
```

- The **data type** of arguments must be **appropriate**

```
order_summary("120", 3, 0.1)
```

- When should we use **keyword arguments** (e.g., price=120)?

```
order_summary(price=120, quantity=3, discount_rate=0.1)

order_summary(discount_rate=0.1, price=120, quantity=3)

order_summary(120, quantity=3, discount_rate=0.1)

order_summary(quantity=3, 120, discount_rate=0.1)
```



Default Parameter Values

- You can assign **default values** to parameters.
- If no argument is provided when calling the function, the default value is used.

```
def my_function(name = "friend"):
    print("Hello", name)

my_function("Emil")
my_function("Tobias")
my_function()
my_function("Linus")
```

Keywords Arguments

Arguments can be passed using the `key=value` syntax.

- With keyword arguments, the order does not matter.
- In Python documentation, keyword arguments are often abbreviated as `kwargs`.

```
def my_function(animal, name):  
    print("I have a", animal)  
    print("My", animal + "'s name is", name)  
  
my_function(animal = "dog", name = "Buddy")  
my_function(name = "Buddy", animal = "dog")
```

Positional Arguments

Arguments passed *without keywords* are called **positional arguments**.

- They must follow the **correct parameter order**.
- Changing the order may **change the result or cause errors**.

```
def my_function(animal, name):  
    print("I have a", animal)  
    print("My", animal + "'s name is", name)  
  
my_function("dog", "Buddy")  
my_function("Buddy", "dog")  
✓ 0.0s
```

I have a dog
My dog's name is Buddy
I have a Buddy
My Buddy's name is dog

Mixing Positional and Keyword Arguments

- You can combine positional and keyword arguments in a function call.
- **Rule:** Positional arguments must appear **before** keyword arguments.

```
def my_function(animal, name, age):  
    print("I have a", age, "year old", animal, "named", name)  
  
my_function("dog", name = "Buddy", age = 5)
```

Further Reading

For keyword-only arguments, positional-only arguments, and their combination, refer to [W3Schools Python tutorial](https://www.w3schools.com/python/python_function_arguments.asp).

Arguments before / are positional-only, and arguments after * are keyword-only.

Arbitrary Positional Arguments `*args`

- Sometimes we do not know **how many arguments** will be passed to a function.
- Use `*args` to accept **multiple positional arguments**.

Explanation

- `*args` collects arguments into a **tuple**.
- The function can receive **any number** of inputs.

```
def my_function(*numbers):  
    if len(numbers) == 0:  
        return None  
    max_num = numbers[0]  
    for num in numbers:  
        if num > max_num:  
            max_num = num  
    return max_num  
  
print(my_function(3, 7, 2, 9, 1))
```

```
def my_function(*args):  
    if len(args) == 0:  
        return None  
    max_num = args[0]  
    for num in args:  
        if num > max_num:  
            max_num = num  
    return max_num  
  
print(my_function(3, 7, 2, 9, 1))
```

Any difference?

Arbitrary arguments are often abbreviated as `*args` in Python documentation.

Arbitrary Keyword Arguments `**kwargs`

If the number of keyword arguments is unknown, use `**` *before the parameter name*.

- `**kwargs` allows a function to accept **any number of keyword arguments**.
- Inside the function, `kwargs` becomes a **dictionary** containing these arguments.

In Python documentation, arbitrary keyword arguments are often abbreviated as `**kwargs`.

```
def my_function(**myvar):  
    print("Type:", type(myvar))  
    print("Name:", myvar["name"])  
    print("Age:", myvar["age"])  
    print("All data:", myvar)  
  
my_function(name = "Tobias", age = 30, city = "Bergen")
```

Guess the output

`**myvar` → `**kwargs`

Combine Regular Arguments, `*args`, `**kwargs`

- You can combine regular parameters, `*args`, and `**kwargs` in the same function.
- Order rule:
 - Regular parameters
 - `*args` (variable positional arguments)
 - `**kwargs` (variable keyword arguments)
- Regular parameters must appear **before** `*args` and `**kwargs`.

```
def my_function(title, *args, **kwargs):  
    print("Title:", title)  
    print("Positional arguments:", args)  
    print("Keyword arguments:", kwargs)  
  
my_function("User Info", "Emil", "Tobias", age = 25, city = "Oslo")
```

Calling and Using Functions

Good practice → Read the documentation to understand the arguments, behavior, and return value.

An example

- To call a function, write its **name followed by parentheses**.
- Types of functions
 - Built-in functions: `input()`, `print()`, `len()`
 - Functions defined in your program: `order_summary()`
 - Class methods: `a="python"`, `a.capitalize()`, `"python".capitalize()`
 - Functions from external modules: `import pandas as pd`, `pd.read_csv()`
- When using a function, consider:
 - **Inputs:** required arguments (positional/keyword), number, and data types
 - **Process:** how the function transforms inputs (sometimes)
 - **Output:** what the function returns

Lambda Functions

- A lambda function is a small **anonymous function** defined without using `def`.
- Syntax

`lambda arguments: expression`
- Characteristics
 - Can take any number of arguments
 - Contains a single expression
 - Returns the result automatically
- Typical used for short, simple operations, often inside other functions (e.g., sorting or data processing).

```
square = lambda x: x * x
print(square(4))
```

```
add = lambda a, b: a + b
print(add(3, 5))
```

```
students = [("Emil", 25), ("Tobias", 22), ("Linus", 28)]
sorted_students = sorted(students, key=lambda x: x[1])
print(sorted_students)
```

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
odd_numbers = list(filter(lambda x: x % 2 != 0, numbers))
print(odd_numbers)
```

Additional Function Concepts (Advanced Topics)

- [Local vs. Global Scope](#)
 - Variables defined **inside** a function are local and only accessible **within that function**.
 - Variables defined **outside** functions are global and can be accessed **throughout the program**.
- [Decorators](#)
 - A decorator is a function that **modifies or extends the behavior of another function** without changing its code.
- [Recursion](#)
 - A technique where a function **calls itself** to solve a problem by breaking it into smaller subproblems.
- [Generators](#)
 - Special functions that yield values one at a time using `yield`, allowing efficient iteration over large data sequences.

In-Class Exercise

Consider the following Python function:

```
def process_scores(*scores, bonus=0):  
    total = sum(scores)  
    average = total / len(scores)  
    final_score = average + bonus  
    return average, final_score  
  
print(process_scores(80, 90, 100, bonus=5))
```

- 1) What will be printed? What is the average score? Final score?
- 2) What is the data type of scores inside the function? What values does it contain?
- 3) What happen if we run: `process_scores()`?
 - A. Works normally
 - B. Returns 0
 - C. Causes an error

Summary

- A function is a reusable block of code that performs a specific task.
- Defining and Calling Functions
 - Use `def` to define a function
 - Call a function using `function_name(arguments)`
- Arguments
 - Positional arguments: order matters
 - Keyword arguments: passed using `key=value`
 - Default parameters: provide fallback values
 - `*args` collects multiple positional arguments (tuple)
 - `**kwargs` collects multiple keyword arguments (dictionary)
- Lambda function: small anonymous functions for simple operations

Q&A

CBPM B214

dchen@kean.edu

Tue & Thu 11:15 – 12:30 | Wed 14:00 – 16:30